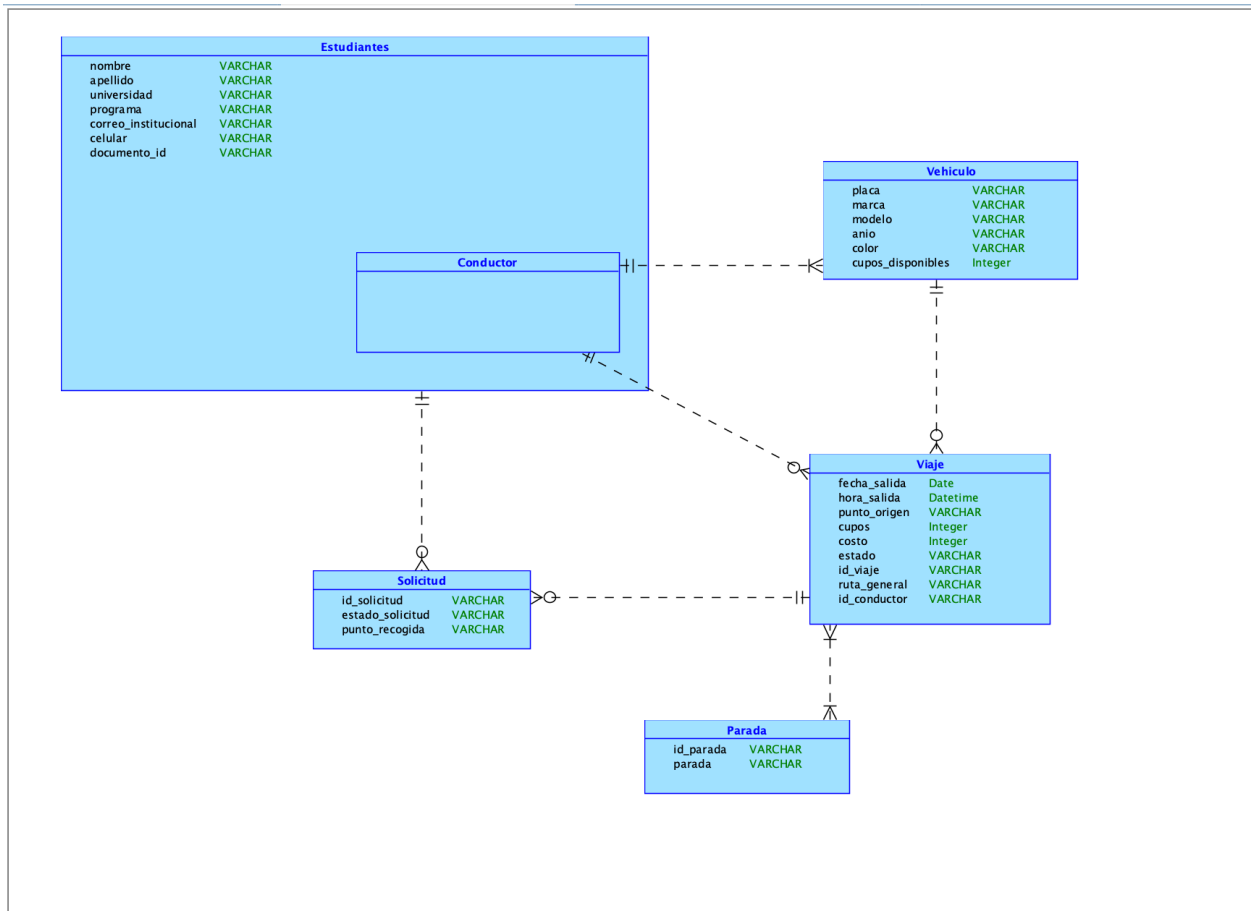
Sofia Muñoz 202521447 - Mariana Gaviria 202520483 - Simon Velasquez 202324865

Análisis y revisión del modelo de datos

Al ajustar el modelo conceptual E/R teniendo en cuenta los comentarios de la monitora llegamos a que teniamos que hacer los siguientes cambios:

1. Eliminamos la entidad “Pasajero” porque ya existe la entidad Estudiantes y se entiende que un pasajero automaticamente es un estudiante. De esta forma evitamos repetir informacion. Al hacer este cambio, Solicitud ahora se relaciona directamente con Estudiantes.
2. La relacion entre Viaje y Parada ahora es de 1..* a 1…* porque un viaje puede tener varias paradas y una misma parada puede estar en diferentes viajes.
3. En la entidad Viajes se agregó el atributo de id_conductor porque de esta forma podemos saber qué conductor hace un viaje específico.



Teniendo en cuenta estos cambios, actualizamos nuestro documento del modelo relacional:

1. Eliminamos la tabla de Pasajero y en las FK que referenciaban a pasajero, lo cambiamos por estudiantes.
2. Al hacer el cambio entre viaje y parada, agregamos otra tabla llamada Viaje_parada porque era necesaria una tabla intermedia para conectar los viajes con sus paradas, y de este forma un viaje puede tener varias paradas y una misma parada puede estar en diferentes viajes. Haciendo esto, evitamos repetir información. Esta tabla entonces tiene id_viaje y id_parada como PK; id_viaje tiene como FK viaje referenciando id_viaje y id_parada tiene como FK parada referenciando id_parada.
3. En la tabla de viajes se agregó id_conductor como FK de conductor referenciando a documento_id

Estudiantes

documento_id	nombre	apellido	programa	universidad	correo_institucional	celular
PK	NN	NN	NN	NN	NN	NN

Conductor

documento_id
PK, FK(Estudiantes, documento_id)

Viaje_parada

id_viaje	id_parada
PK, FK(viaje, id_viaje)	PK, FK(parada, id_parada)

Vehiculo

id_conductor	placa	marca	modelo	anio	color	cupos_disponibles
FK(Conductor, documento_id)	PK	NN	NN	NN	NN	NN

Viaje

id_viaje	fecha_salida	hora_salida	punto_origen	cupos	costo	estado	ruta_general	placa	id_conductor
PK	NN	NN	NN	NN	NN	NN	NN	FK(Vehiculo, placa)	FK(Conductor, documento_id)

Solicitud

id_solicitud	estado_solicitud	id_pasajero	id_viaje	punto_recogida
PK	NN	FK(Estudiantes, documento_id)	FK(Viaje, id_viaje)	NN

Parada

id_parada	parada
PK	NN

Todas las relaciones del modelo están en la forma normal BCNF porque en cada tabla las dependencias funcionales las determina una superllave.

Diseño de sentencias SQL:

RFC1 – Consultar demanda por localidad para un período de tiempo específico:
Esta consulta permite visualizar la cantidad de solicitudes de cupo agrupadas por localidad en un intervalo de fechas definido. Utilizamos las tablas Viaje y Solicitud relacionadas por medio del atributo id_viaje usando un JOIN ON ya que esto nos permite contar las solicitudes que están asociadas a un viaje. Adicional a esto, usamos un CROSS JOIN para poner un filtro de intervalo de fechas. Nosotros escogimos poner del 1 al 31 de Octubre del 2026. Finalmente, para contar las solicitudes hicimos uso de COUNT, lo agrupamos por punto_origen que es la localidad y las organizamos de mayor a menor cantidad.

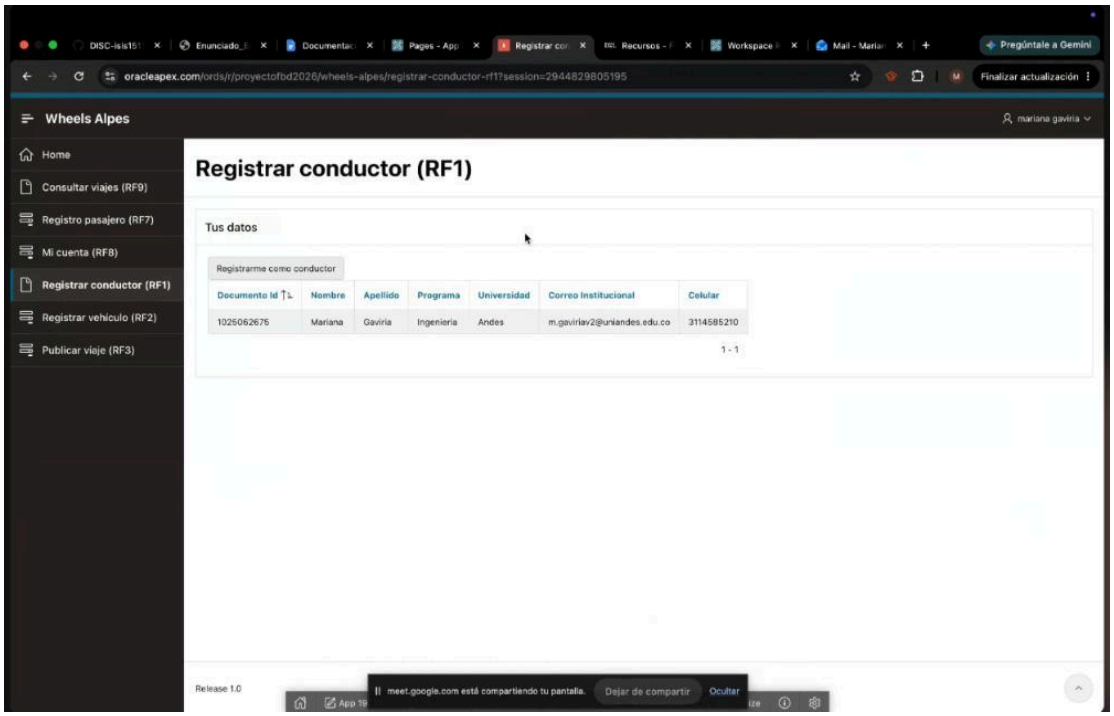
RFC2 – Consultar ingresos estimados por localidad y período:
Esta consulta permite visualizar los ingresos totales estimados generados por la plataforma en un rango de fechas, agrupados por localidad. Utilizamos las tablas Viaje y Solicitud relacionadas por medio del atributo id_viaje usando un JOIN ON ya que esto nos permite saber las solicitudes aceptadas de un viaje**.** Adicional a esto usamos un CROSS JOIN para poner un filtro de rango de fechas y también filtramos los viajes con estado “completado” y las solicitudes con estado “aceptada”. Finalmente, usamos SUM para sumar el costo de los viajes por cada solicitud aceptada, luego lo agrupamos por localidad y organizamos los ingresos estimados de mayor a menor.

RFC3 – Consultar viajes por conductor a través del tiempo:
Esta consulta muestra la evolución de los viajes realizados por cada conductor durante un período determinado. Se puede filtrar por conductor específico, localidad y rango de fechas. Como la tabla viaje ya tiene toda la informacion necesaria como el id_conductor, el punto de origen, el estado del viaje y la fecha de salida, no tuvimos necesidad de unirla con otra tabla. Utilizamos un CROSS JOIN para poner un filtro para conductor, rango de fechas y localidad. En el código decidimos poner NULL OR para hacer que los filtros sean opcionales, ya que cuando se usa esto, se muestran todos los registros. Finalmente, usamos COUNT para contar los viajes completados y TRUNC para agruparlos por fecha sin contar la hora.

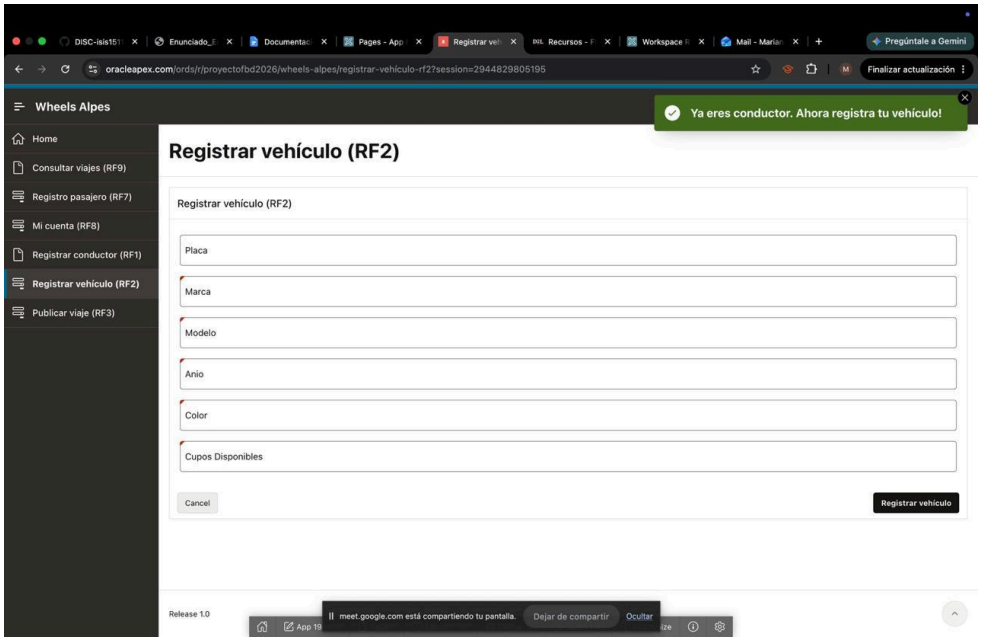
Escenarios de prueba

RF1 – Registrar conductor:
Prueba exitosa:
Objetivo: Registrar la información personal del conductor (nombre, apellidos, universidad, programa que cursa, correo electrónico institucional, número de celular y documento de identidad).
Resultado esperado: la aplicacion debe permitir al estudiante registrarse como conductor.
Resultado obtenido: Al seleccionar “registrarme como conductor” la aplicación mostró un mensaje diciendo “Ya eres conductor. Ahora registra tu vehículo!”

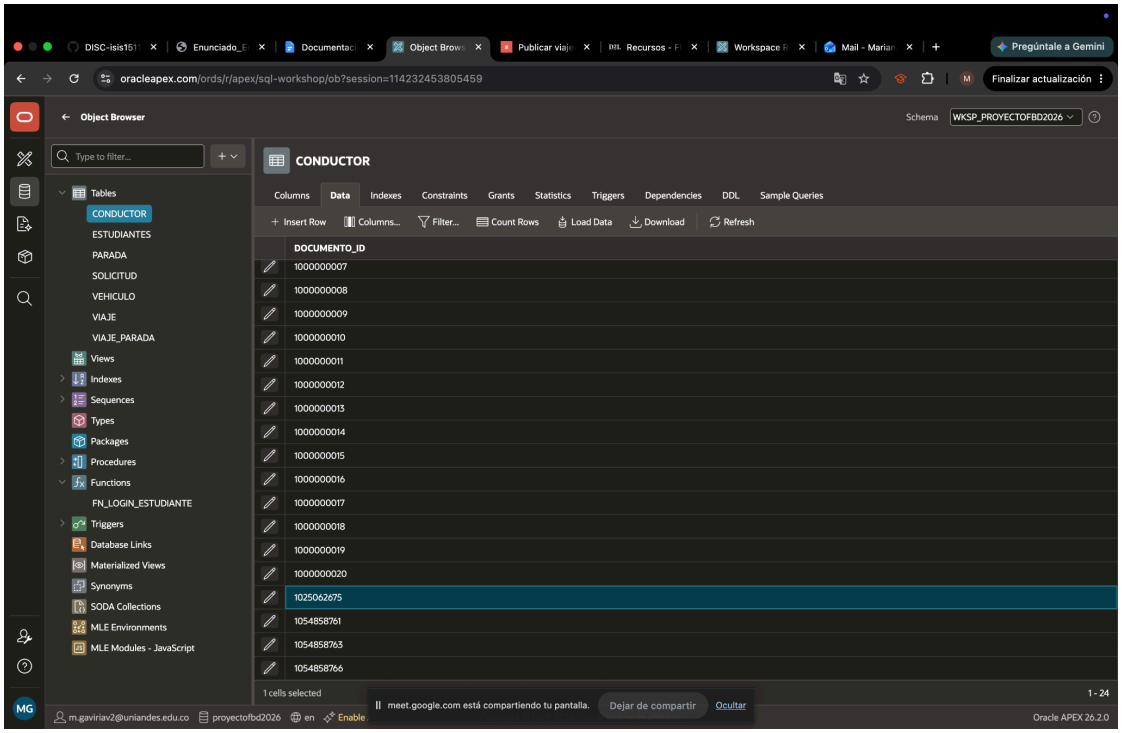
Prueba fallida:
Objetivo: Probar que un estudiante no se pueda registrar dos veces como conductor.
Resultado esperado: El estudiante no se puede registrar.
Resultado obtenido: El boton que da la opcion al estudiante de registrarse como conductor ya no aparece, de manera que no se puede volver a registrar.



En esta foto se puede ver el boton que permite al estudiante registrarse como conductor.



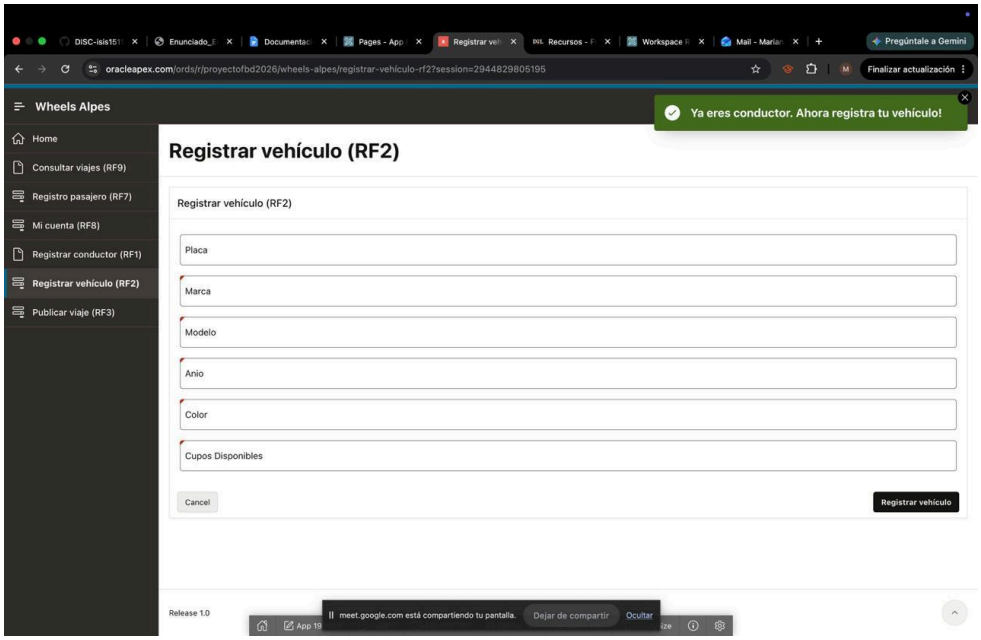
En esta foto se puede observar que el registro fue exitoso, que el estudiante ya es conductor y que ahora debe registrar su vehículo.



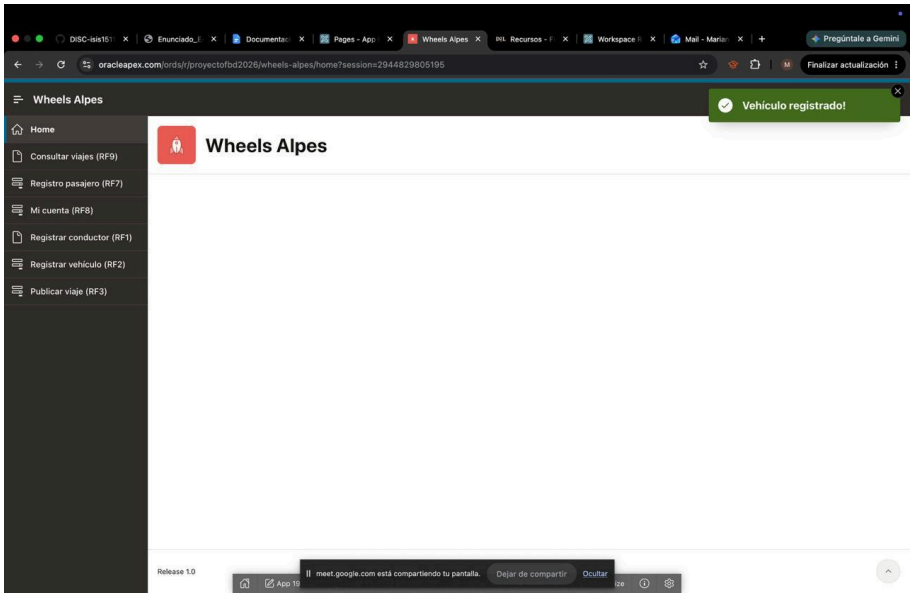
En esta foto se puede evidenciar como el documento del estudiante registrado solo aparece una vez en la tabla de conductor.

RF2 – Registrar vehículo:
Prueba exitosa:
Objetivo: Registrar un vehículo con placa, marca, modelo, año, color y número de cupos disponibles.
Resultado esperado: A la hora de llenar el formulario con la información, el vehículo debe ser registrado exitosamente.
Resultado obtenido: Aparece un mensaje avisando al estudiante que el vehículo fue registrado correctamente.

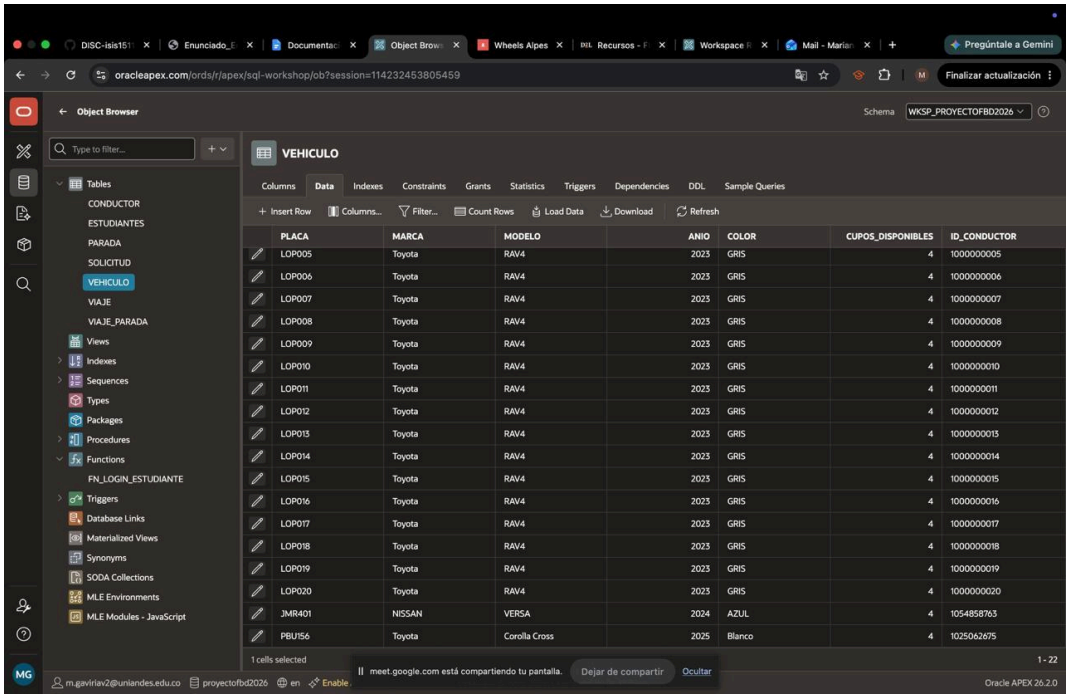
Prueba fallida:
Objetivo: Que no se pueda registrar un dos vehiculos con la misma placa, así el resto de la informacion sea diferente.
Resultado esperado: al estudiante no le deben permitir registrar dos vehiculos con la misma placa.
Resultado obtenido: En el momento en el que se diligencia el formulario y se identifica que hay una placa repetida, se muestra un mensaje de error.



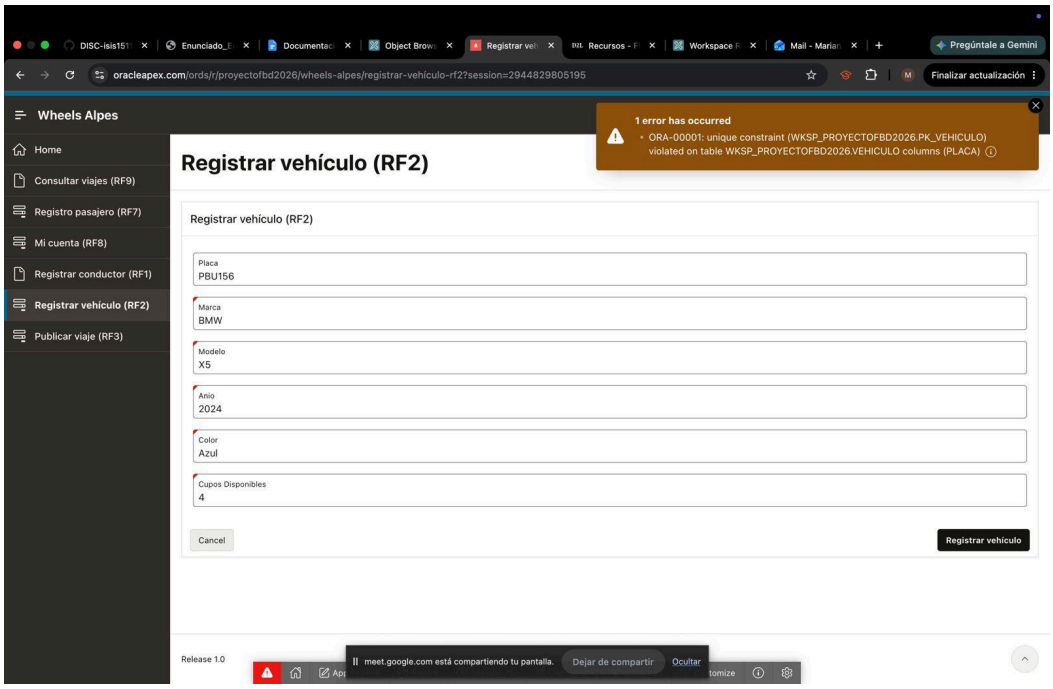
En esta foto se muestra el formulario para registrar un vehiculo



En esta foto se evidencia el mensaje de que el vehiculo fue registrado exitosamente.



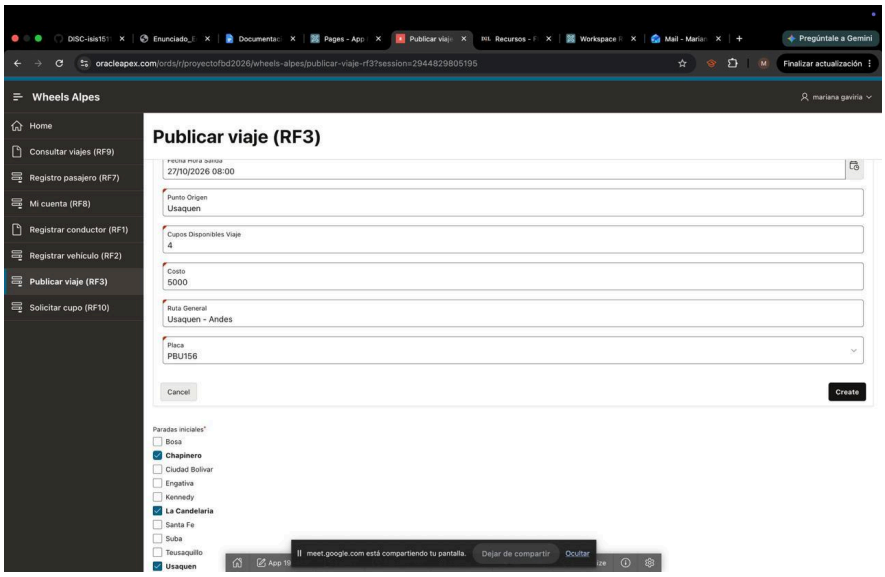
En esta foto se puede ver que el vehículo quedó registrado una vez con la placa PBU156.



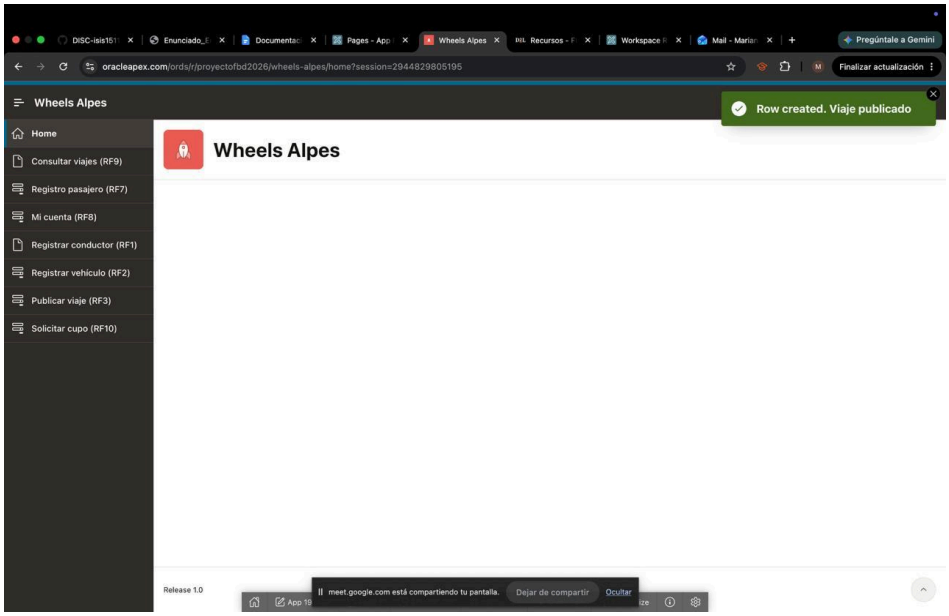
En esta foto se evidencia qué es lo que ocurre a la hora de intentar registrar un vehículo con una placa que ya estaba registrada.

RF3- Publicar viaje:
Prueba exitosa:
Objetivo: Crear un nuevo viaje indicando fecha, hora de salida, punto de origen, paradas iniciales, ruta general y tarifa. Adicionalmente debe asociar un vehículo a dicho viaje.
Resultado esperado: El estudiante debe poder registrar un viaje nuevo con su vehículo y con las paradas respectivas.
Resultado obtenido: En el momento en el que se llenó el formulario, aparece un mensaje de que el viaje ha sido publicado.

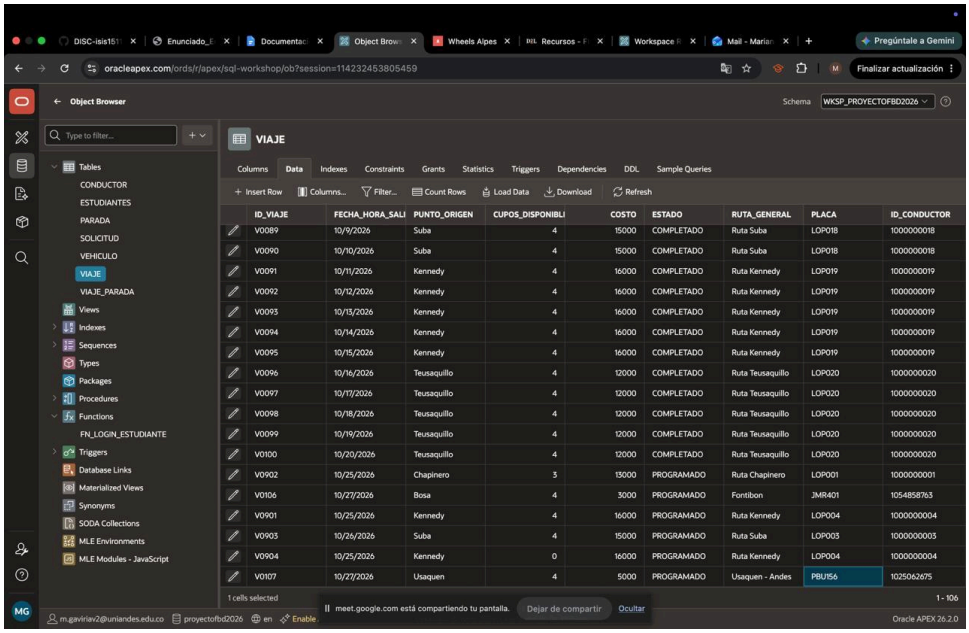
Prueba Fallida:
Objetivo: Demostrar que la aplicacion no debe dejar registrar un viaje con una cantidad negativa de cupos
Resultado esperado: No dejar al estudiante registrar su viaje
Resultado obtenido: Mostró un mensaje de error explicando brevemente cual es la causa del problema.



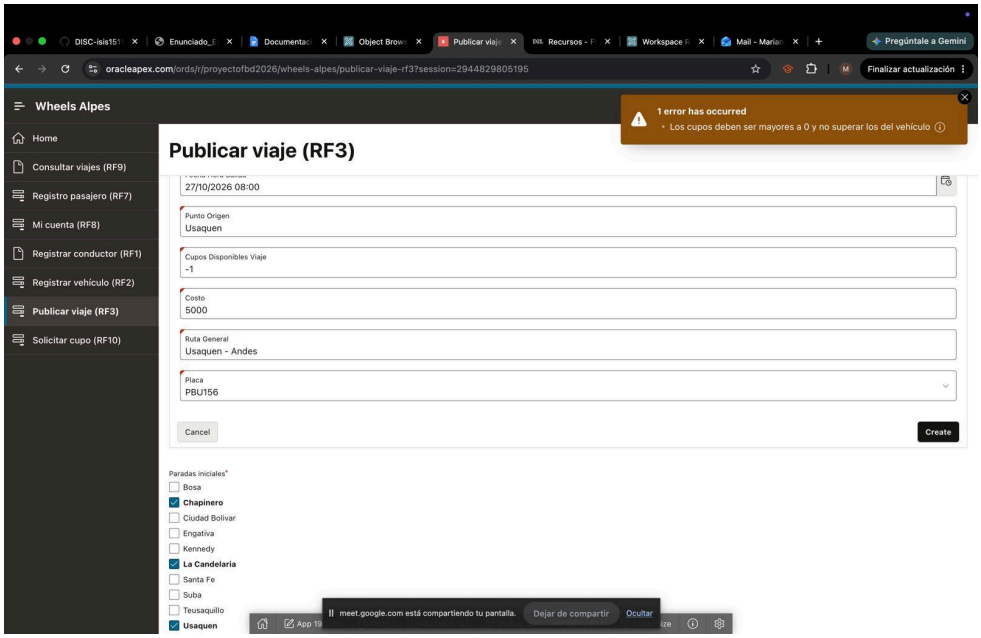
En esta foto se puede ver la información que el estudiante debe llenar para poder publicar su viaje.



En esta foto se ve que el viaje se publicó correctamente.



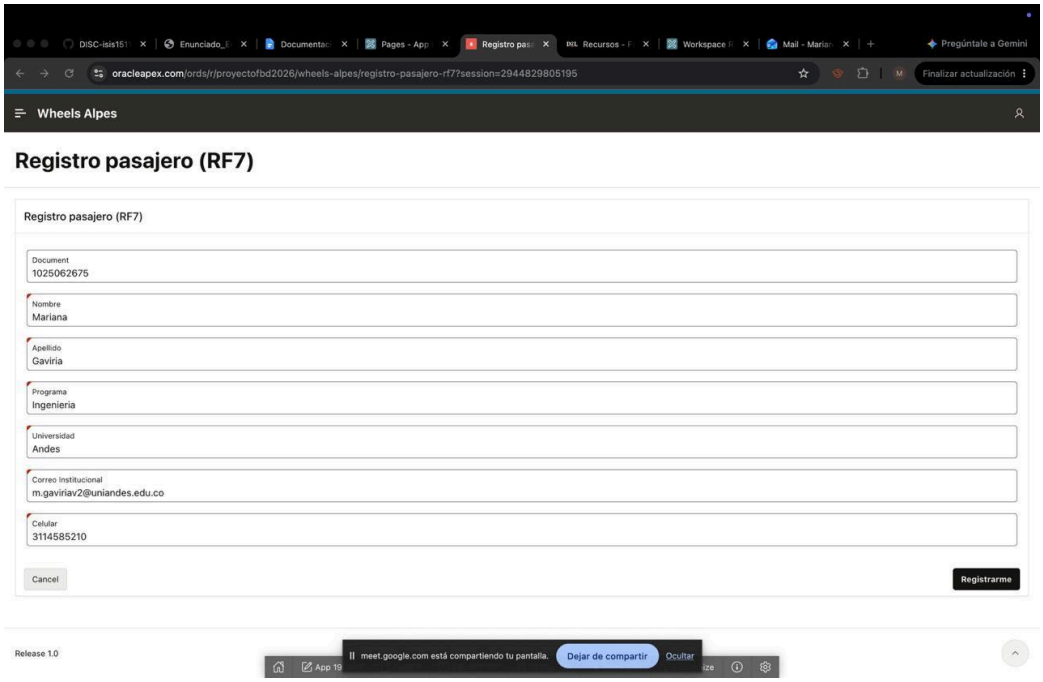
En esta foto se puede ver que el viaje quedó registrado solo una vez.



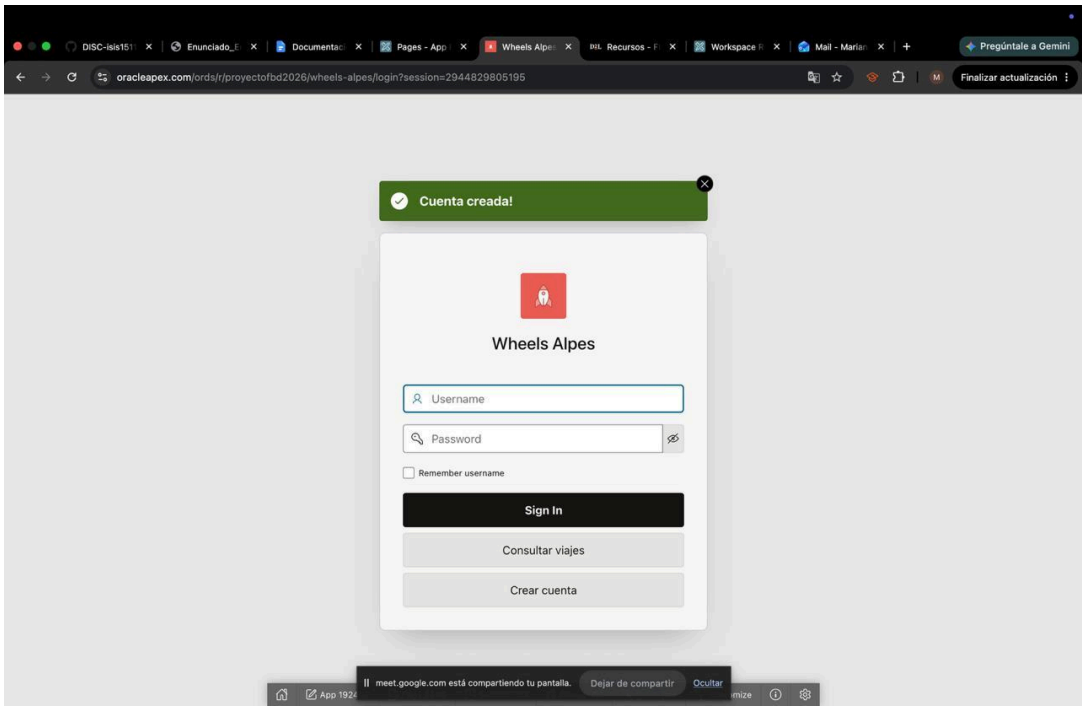
En esta foto se puede evidenciar lo que sucede cuando un estudiante quiere registrar un viaje con un número de cupos negativo.

RF7 – Creación de cuenta pasajero:
Prueba exitosa:
Objetivo: Registrar la información personal del pasajero (nombre, apellidos, universidad, programa que cursa, correo electrónico institucional, número de celular y documento de identidad).
Resultado esperado: que un estudiante a la hora de llenar el formulario quede registrado exitosamente
Resultado obtenido: el estudiante pudo registrarse

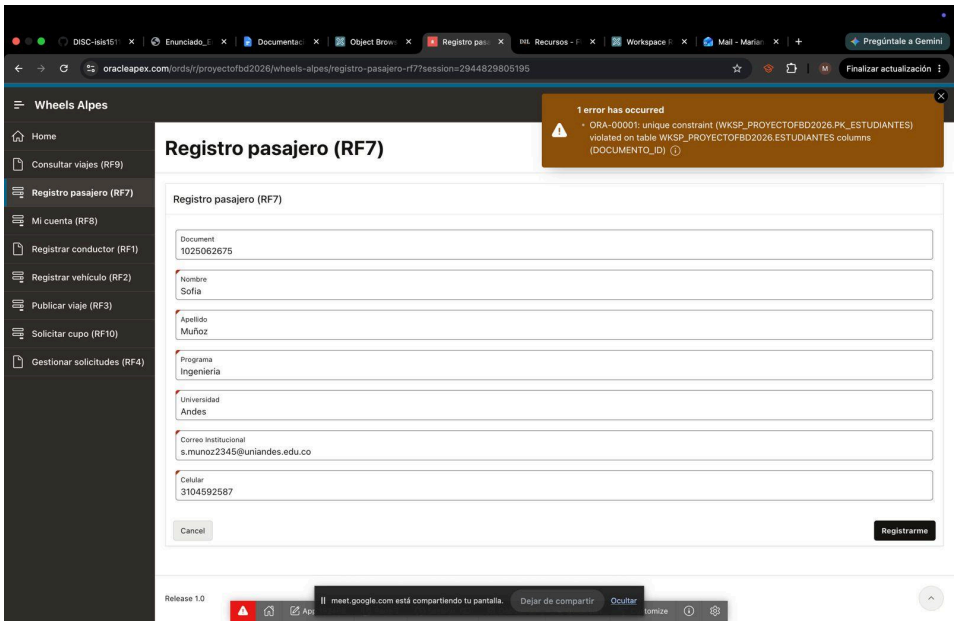
Prueba fallida:
Objetivo: Demostrar que un estudiante no se puede registrar si utiliza un documento que ya está registrado.
Resultado esperado: Que cuando el estudiante intente registrarse con un documento que ya está registrado aparezca error.
Resultado obtenido: Cuando el estudiante intentó registrarse con un número de documento que ya estaba registrado apareció un mensaje de error y el registro no fue exitoso.



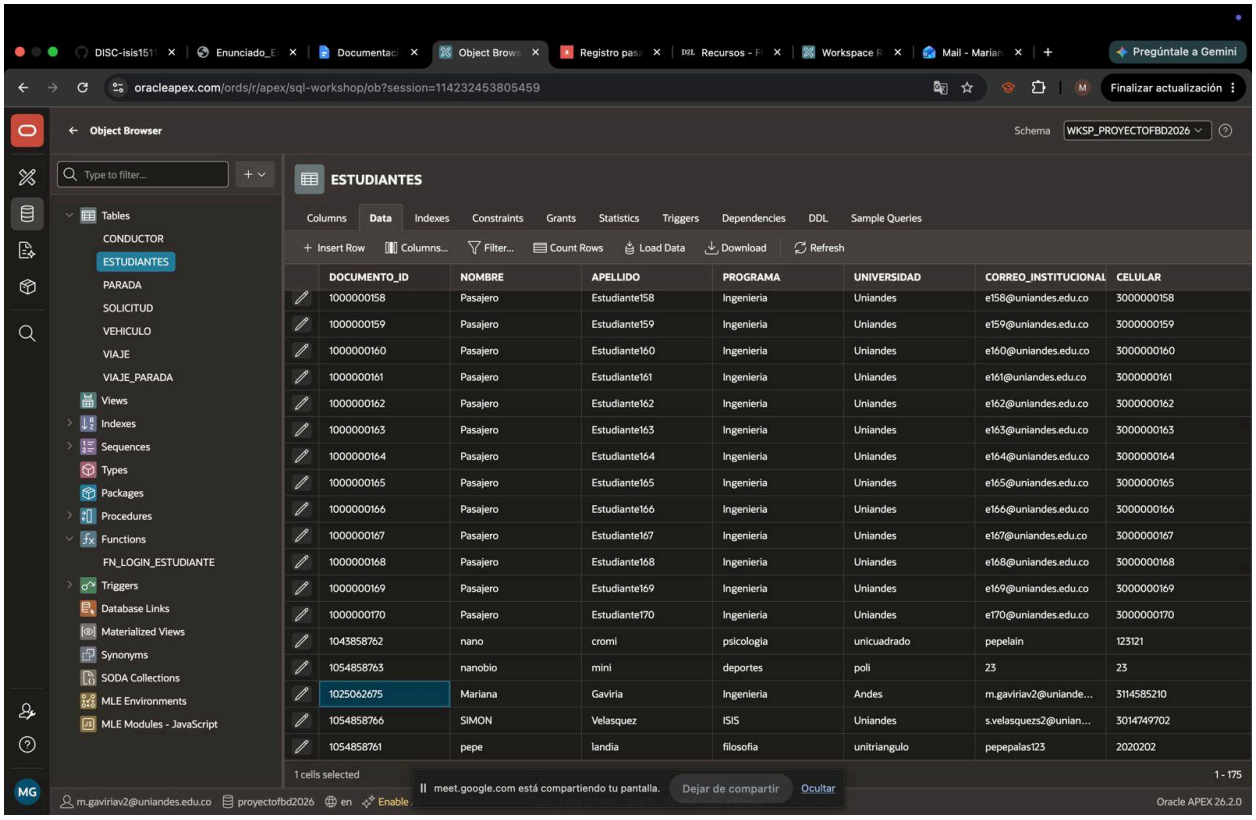
En esta foto se puede ver la informacion que fue inscrita a la hora de crear un nuevo pasajero.



En esta foto se puede ver que a la hora de registrarse exitosamente, se debe redirigir a esto en donde puede ingresar su usuario y su contraseña.



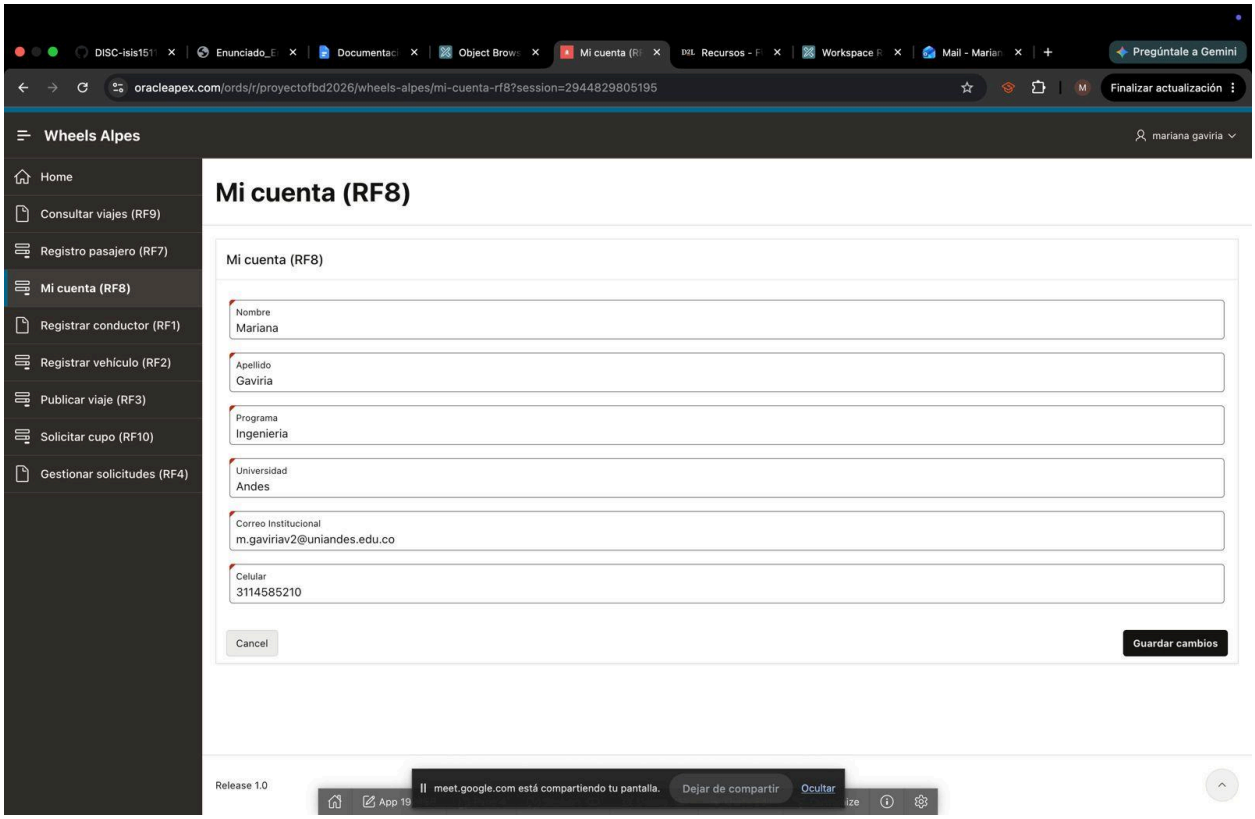
En esta foto se puede ver que cuando el estudiante intentó registrarse con un documento que ya estaba registrado, le salió un error y no se pudo registrar.



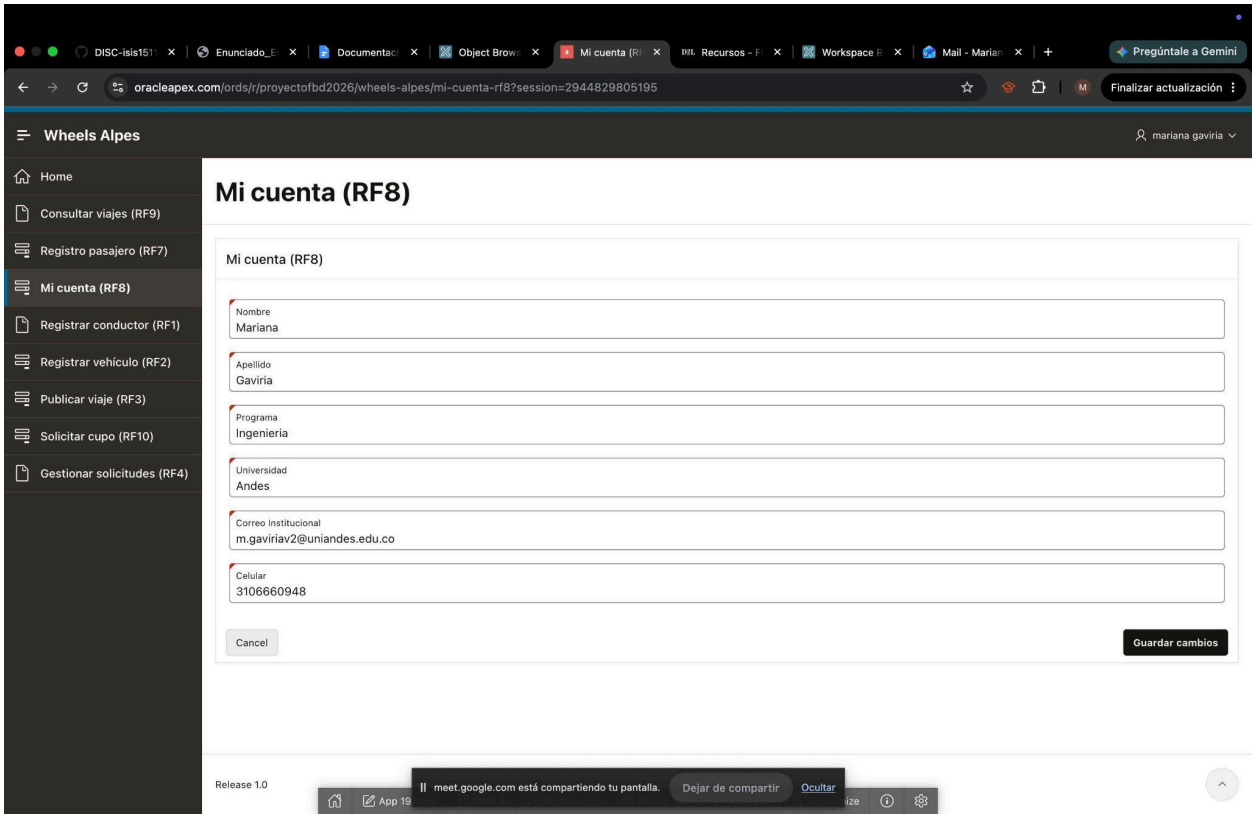
En esta foto se puede evidenciar que solamente hay un registro por cedula.

RF8 – Gestión de cuenta pasajero:
Prueba exitosa:
Objetivo: Modificar sus datos personales (nombre, apellidos, universidad, programa que cursa, correo electrónico institucional, número de celular y documento de identidad).
Resultado esperado:
Resultado obtenido:

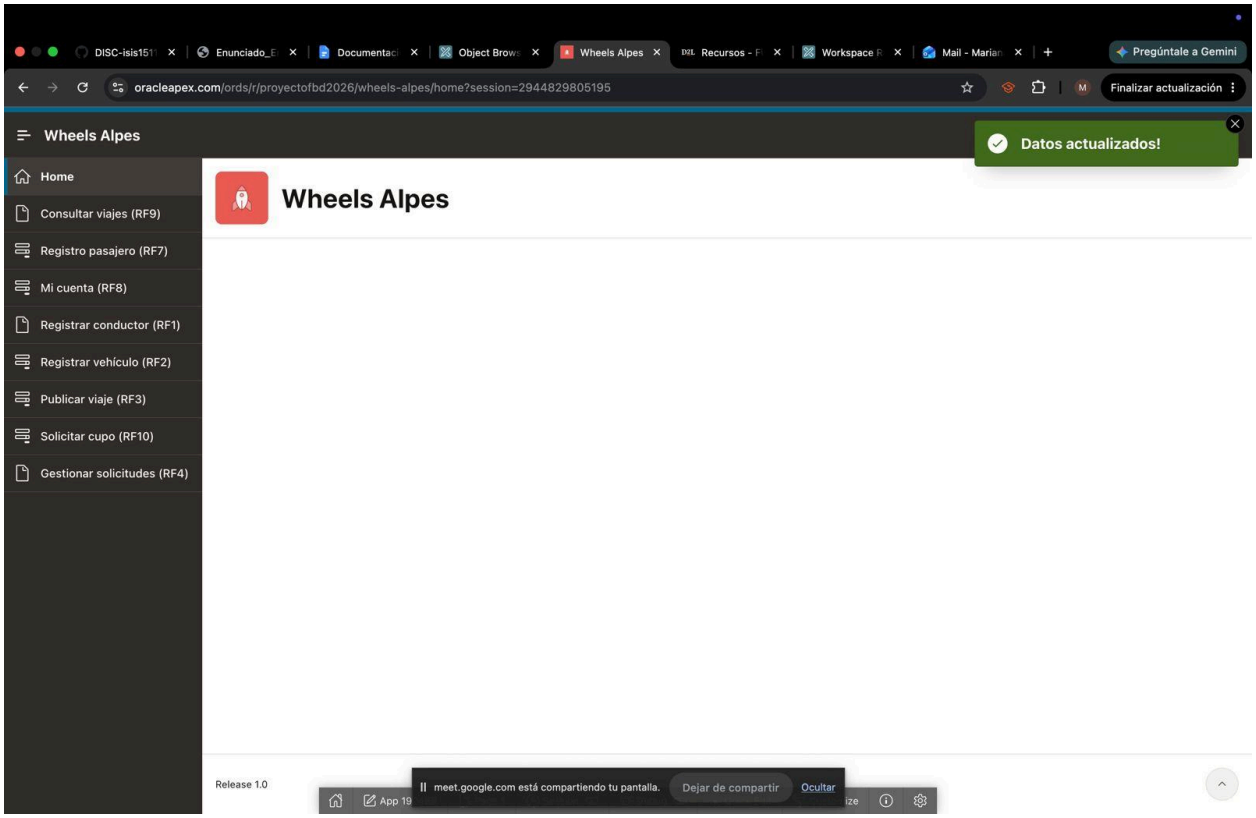
Prueba fallida:
Objetivo: Demostrar que no se puede dejar un campo obligatorio vacío y todos los campos deben tener información válida.
Resultado esperado: Que la aplicación no acepte los cambios cuando hay un campo obligatorio vacío.
Resultado obtenido: Al momento de modificar datos y dejar un campo obligatorio vacío, la aplicación lo rechazó y salió un mensaje de error.



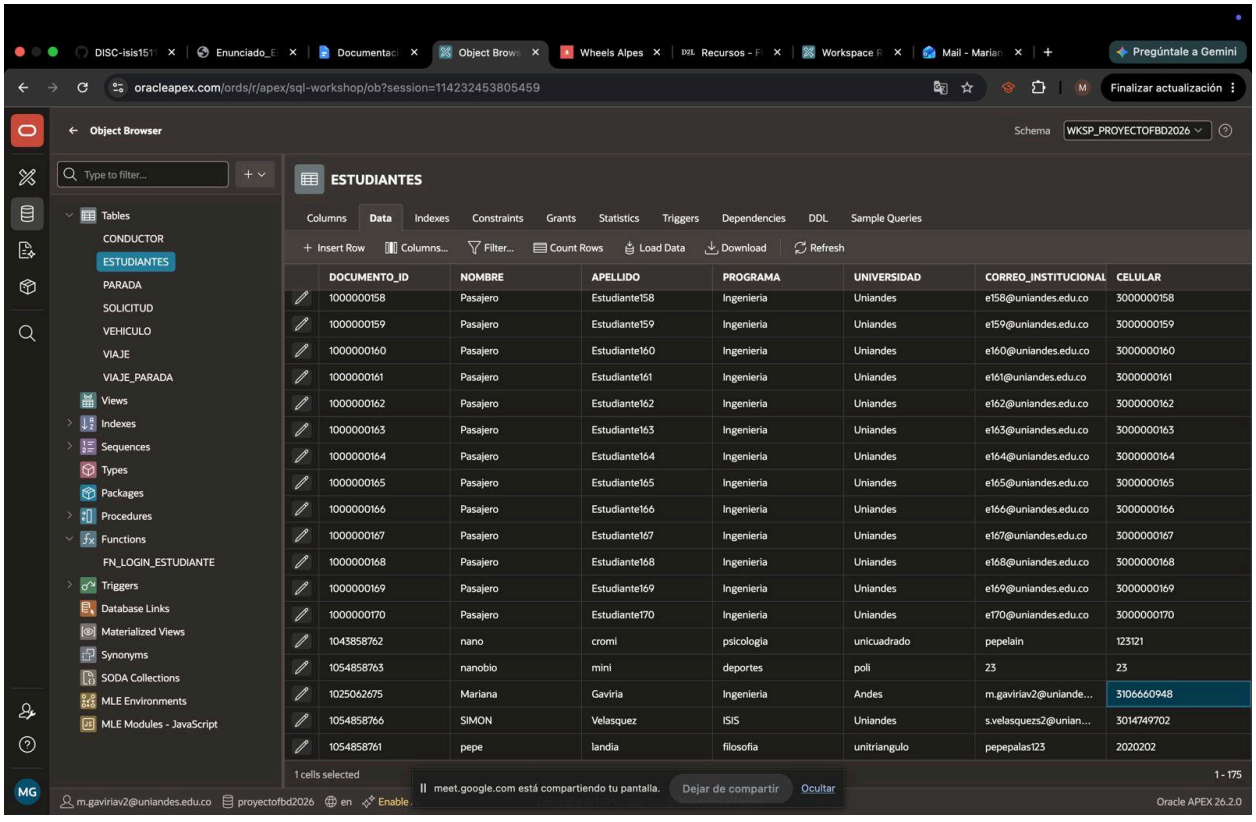
En esta foto se pueden ver los campos que cada cuenta debe tener.



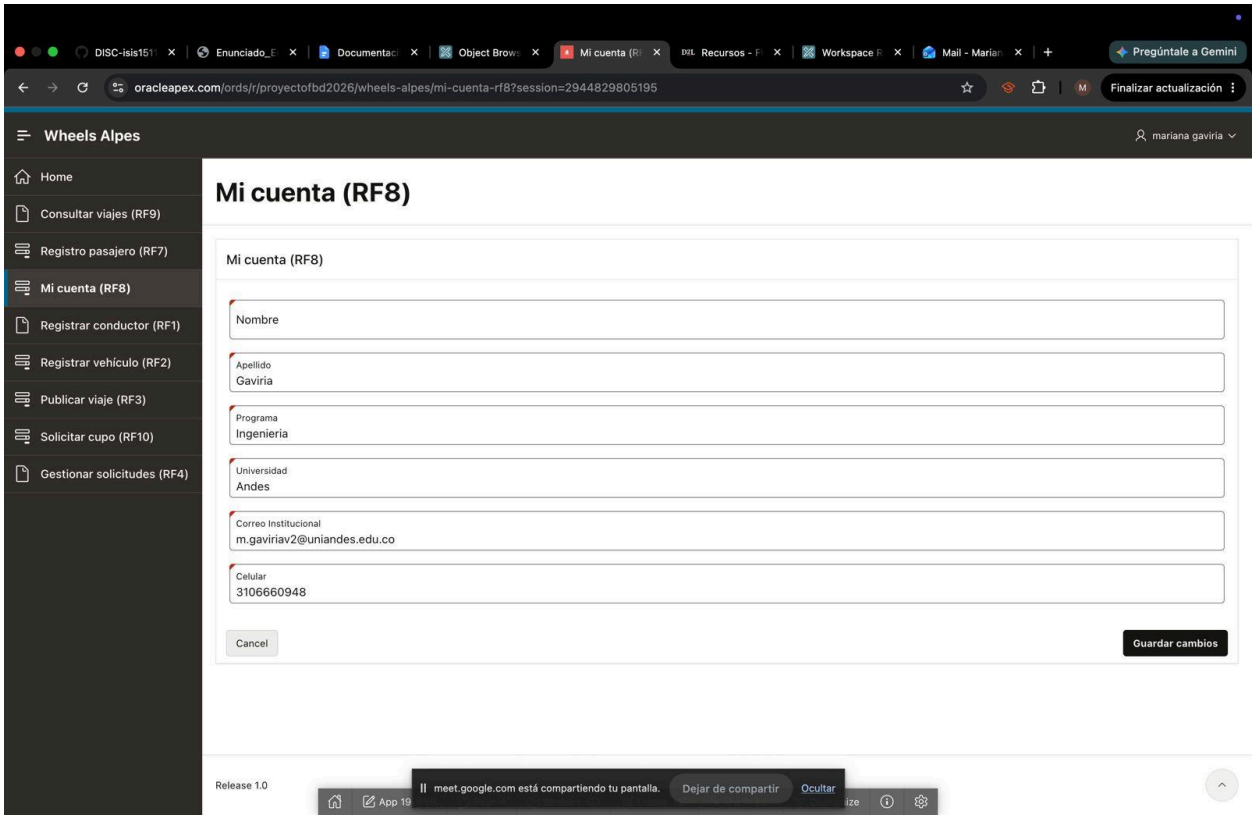
En esta foto, decidimos cambiar informacion para demostrar que la aplicacion efectivamente guarda estos cambios. En este caso modificamos el número de celular.



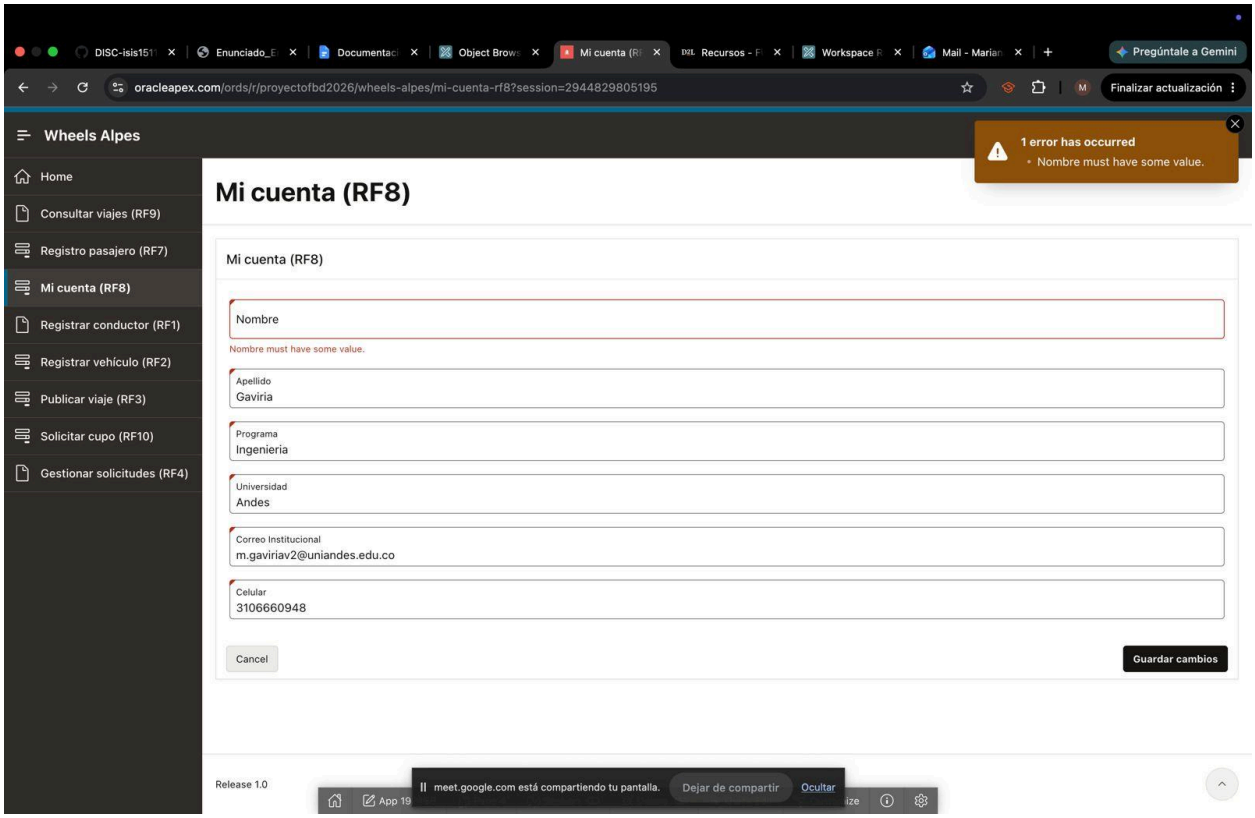
En esta imagen se evidencia el mensaje que aparece a la hora de modificar la informacion



En esta imagen se puede ver en la parte resaltada que se cambio la informacion del estudiante de manera adecuada.



En esta imagen estamos haciendo la prueba fallida en la que dejamos un campo vacío.



En esta imagen se evidencia lo que sucede a la hora de dejar un campo de la cuenta de cada estudiante vacio.